

TELOS

Hyperlocal Peer-to-Peer Sharing Economy Platform

Technical & Product Documentation

Telos (from the Greek 'telos', meaning purpose or end goal) is a platform that lets neighbours share, lend, borrow, and rent everyday items and skills within their immediate geographic community. This document describes the product vision, system architecture, technology choices, data model, API surface, security posture, and delivery roadmap.

Document Overview

Project: Telos - Hyperlocal P2P Sharing Economy Platform

Backend: Java 21, Spring Boot 4.1, PostgreSQL + PostGIS, Hibernate Spatial

Frontend: Single-page web application (to be implemented)

Audience: Engineers, product stakeholders, and new contributors

Status: Foundation / scaffolding phase

Version 1.0

Generated automatically from the project repository.

Table of Contents

1. Executive Summary	3
2. Vision & Problem Statement	3
2.1 The problem	3
2.2 The vision	3
3. Solution Overview	3
3.1 Core user journeys	4
4. Key Features	4
4.1 Minimum viable product (MVP)	4
4.2 Planned enhancements	4
5. System Architecture	4
5.1 High-level components	4
5.2 Backend layering	5
5.3 Why this shape	5
6. Technology Stack	5
7. Data Model	6
7.1 Entities	6
7.2 Relationships	6
7.3 Example spatial query	6
8. API Design	7
8.1 Representative endpoints	7
8.2 Conventions	7
9. Frontend	7
9.1 Expected responsibilities	8
10. Security, Privacy & Trust	8
11. Building & Running	8
11.1 Prerequisites	8
11.2 Common commands	8
12. Repository Structure	9
13. Delivery Roadmap	9
Phase 1 - Foundation (current)	9
Phase 2 - Core marketplace	9
Phase 3 - Transactions & trust	9
Phase 4 - Engagement	9
Phase 5 - Growth	9
14. Glossary	9
15. Conclusion	9

1. Executive Summary

Telos is a hyperlocal, peer-to-peer (P2P) sharing economy platform. It connects neighbours so they can lend, borrow, rent, and exchange physical items and services within walking or short-driving distance of one another. Instead of every household buying tools, equipment, and occasionally-used goods, Telos turns a neighbourhood into a shared inventory.

The platform's defining characteristic is proximity. Every listing, search, and match is anchored to geographic location. This is enforced at the database layer using PostgreSQL with the PostGIS extension and accessed through Hibernate Spatial, so distance and radius queries are first-class, indexed operations rather than application-side calculations.

In one sentence

Telos helps communities access more while owning less, by making the things and skills around them easy to discover, trust, and borrow.

2. Vision & Problem Statement

2.1 The problem

- Underused assets: drills, ladders, pressure washers, and party supplies sit idle in garages for 99% of their life.
- Overconsumption: people repeatedly buy items they need only once or twice, creating cost and waste.
- Weak local ties: neighbours often do not know what help or resources exist a few doors away.
- Trust friction: existing marketplaces are global and anonymous, which makes lending physical goods feel risky.

2.2 The vision

Telos aims to make borrowing from a neighbour as frictionless as ordering a product online, while rebuilding the social trust that makes sharing sustainable. Success looks like dense, active neighbourhood networks where access replaces ownership for a meaningful share of everyday goods and services.

3. Solution Overview

Telos provides a marketplace built around three primitives: people (users tied to a location), listings (items or services offered), and transactions (a borrow, rent, or exchange agreement between two users). A reputation system and location-aware discovery sit on top of these primitives.

3.1 Core user journeys

- Discover: a user searches for an item or service and sees results ranked by distance.
- Request: the borrower requests an item for a date range; the owner approves or declines.
- Exchange: the two parties meet locally to hand over the item; status updates track the loan.
- Return & review: the item is returned and both parties leave a rating that feeds reputation.

4. Key Features

4.1 Minimum viable product (MVP)

- User registration, authentication, and a profile anchored to a home location.
- Create, edit, and delete listings for items or services, including photos and availability.
- Location-aware search: 'find what is available within X km of me', sorted by distance.
- Borrow / rent request workflow with owner approval and a clear transaction lifecycle.
- Ratings and reviews that build a per-user trust score.

4.2 Planned enhancements

- In-app messaging between borrower and owner.
- Optional deposits and payments for rentals.
- Notifications (email / push) for requests, approvals, and due dates.
- Categories, tags, and saved searches.
- Community groups (a building, a street, a campus) as trust boundaries.

5. System Architecture

Telos follows a conventional, well-understood layered architecture: a client application talks to a stateless REST API, which persists to a spatial relational database. This keeps the system easy to reason about, test, and scale horizontally at the API tier.

5.1 High-level components

Logical architecture



5.2 Backend layering

- Controller layer: REST endpoints, request validation, and DTO mapping.
- Service layer: business rules, transaction lifecycle, and authorization checks.
- Repository layer: Spring Data JPA repositories, including spatial queries.
- Domain layer: JPA entities, including geometry fields managed by Hibernate Spatial.

5.3 Why this shape

A stateless API tier means instances can be added behind a load balancer as traffic grows. Pushing geographic logic into PostGIS keeps proximity queries fast and correct, and avoids loading large candidate sets into application memory just to filter them by distance.

6. Technology Stack

The current stack is drawn directly from the backend build configuration (pom.xml) and project layout.

Language	Java 21 (LTS)
Framework	Spring Boot 4.1.0 (spring-boot-starter-parent)
Web layer	Spring Web MVC (spring-boot-starter-webmvc)
Persistence	Spring Data JPA + Hibernate ORM
Spatial	Hibernate Spatial (geometry types, distance queries)
Database	PostgreSQL (with the PostGIS extension)
Boilerplate	Project Lombok (annotation-based, compile-time only)
Build tool	Apache Maven (Maven Wrapper included: mvnw / mvnw.cmd)
Testing	spring-boot-starter- <i>-test</i> (JPA slice & web MVC tests)

Note on dependencies

The build pins Spring Boot 4.1.0 and Java 21. Hibernate Spatial and the PostgreSQL driver together enable indexed geographic queries, which are central to Telos's hyperlocal model.

7. Data Model

The domain centres on four primary entities. Locations are stored as spatial geometry so the database can index and query them efficiently.

7.1 Entities

User

- id, displayName, email, passwordHash
- homeLocation (Point geometry, SRID 4326) - the anchor for proximity
- reputationScore, createdAt

Listing

- id, owner (User), title, description, type (ITEM or SERVICE)
- location (Point geometry), category, photos
- availabilityStatus, pricePerDay or 'free', createdAt

Transaction

- id, listing, borrower (User), owner (User)
- startDate, endDate, status (REQUESTED, APPROVED, ACTIVE, RETURNED, CANCELLED)
- createdAt, updatedAt

Review

- id, transaction, author (User), subject (User)
- rating (1-5), comment, createdAt

7.2 Relationships

- A User owns many Listings.
- A Listing has many Transactions over its lifetime.
- A Transaction produces up to two Reviews (borrower rates owner and vice versa).

7.3 Example spatial query

PostGIS radius search

```
-- Listings within 5 km of a point, nearest first (PostGIS)
SELECT l.*,
       ST_Distance(l.location, :origin) AS distance_m
FROM   listing l
WHERE  ST_DWithin(l.location, :origin, 5000)
       AND l.availability_status = 'AVAILABLE'
ORDER BY distance_m ASC;
```

8. API Design

The API is REST over HTTPS, exchanging JSON. Endpoints below are the proposed surface for the MVP; they map cleanly onto the entities and user journeys described above.

8.1 Representative endpoints

MVP REST surface

POST	/api/auth/register	Create an account
POST	/api/auth/login	Obtain a session / token
GET	/api/listings	Search listings (?lat,&lng,&radiusKm,&q)
POST	/api/listings	Create a listing
GET	/api/listings/{id}	Listing details
PUT	/api/listings/{id}	Update a listing
DELETE	/api/listings/{id}	Remove a listing
POST	/api/transactions	Request to borrow / rent
PATCH	/api/transactions/{id}	Approve, decline, or update status
GET	/api/transactions	List my transactions
POST	/api/reviews	Leave a review for a transaction

8.2 Conventions

- Standard HTTP status codes (200, 201, 400, 401, 403, 404, 409).
- Pagination via page and size query parameters on list endpoints.
- Consistent error envelope with a machine-readable code and human message.
- DTOs separate the API contract from internal JPA entities.

9. Frontend

The frontend directory is currently a placeholder. The intended client is a single-page web application that consumes the REST API and presents a map- and list-based discovery experience centred on the user's location.

9.1 Expected responsibilities

- Capture the user's location (with consent) and request listings within a radius.
- Display results both as a ranked list and on an interactive map.
- Provide listing creation, request management, and review submission flows.
- Handle authentication state and protect borrower/owner-only actions.

10. Security, Privacy & Trust

- Passwords are stored only as salted hashes; never in plain text.
- All traffic is served over HTTPS; tokens are scoped and expiring.
- Authorization checks ensure users can only modify their own listings and transactions.
- Location privacy: exact coordinates are approximated/obfuscated in public responses; precise meet-up details are shared only after a request is approved.
- Reputation scores and reviews provide social accountability for borrowing physical goods.

Privacy principle

Proximity is the product, but precision is sensitive. Telos should reveal that something is 'nearby' long before it reveals exactly where someone lives.

11. Building & Running

11.1 Prerequisites

- Java Development Kit 21
- PostgreSQL with the PostGIS extension enabled
- Maven (or use the bundled wrapper, mvnw)

11.2 Common commands

Backend commands

```
# from the backend/ directory
./mvnw clean verify      # compile, run tests, and package
./mvnw spring-boot:run   # start the API locally

# enable PostGIS once per database
CREATE EXTENSION IF NOT EXISTS postgis;
```

Database connection settings belong in `src/main/resources/application.properties` (currently minimal). At minimum the datasource URL, username, password, and the Hibernate dialect appropriate for PostGIS should be configured before running against a real database.

12. Repository Structure

Directory tree

```
Telos/
  README.md
  backend/
    pom.xml                # Maven build, Spring Boot 4.1, Java 21
    mvnw, mvnw.cmd         # Maven wrapper
    src/main/java/com/telos/backend/
      BackendApplication.java
    src/main/resources/
      application.properties
    src/test/java/com/telos/backend/
      BackendApplicationTests.java
  frontend/                # placeholder for the web client
  docs/
    generate_documentation_pdf.py
    Telos_Platform_Documentation.pdf
```

13. Delivery Roadmap

Phase 1 - Foundation (current)

- Spring Boot project scaffolding, build pipeline, and spatial database wiring.

Phase 2 - Core marketplace

- User accounts, listings CRUD, and location-aware search.

Phase 3 - Transactions & trust

- Borrow/rent workflow, transaction lifecycle, ratings and reviews.

Phase 4 - Engagement

- Messaging, notifications, and a polished web client.

Phase 5 - Growth

- Payments/deposits, community groups, and mobile clients.

14. Glossary

Hyperlocal	Scoped to a very small geographic area, e.g. a neighbourhood.
P2P	Peer-to-peer; individuals transact directly with each other.
PostGIS	A spatial extension to PostgreSQL for geographic objects and queries.

SRID 4326

The coordinate system for standard WGS84 latitude/longitude.

Telos

Greek for 'purpose' or 'end goal'; the platform's namesake.

15. Conclusion

Telos reframes the things and skills already present in a neighbourhood as a shared, discoverable resource. By anchoring every interaction to location and reinforcing it with reputation, the platform lowers the cost of access while strengthening local trust. The foundation described here - a stateless Spring Boot API over a spatial PostgreSQL database - is intentionally simple, giving the team a clear and scalable path from MVP to a thriving community marketplace.